

Manga Translation Platform (Immich-Inspired)

Product & Technical Architecture Specification v1.0

Vision

Build an open, self-hostable platform that combines:

- Immich-style gallery experience
- OCR and layout detection
- Context-aware AI translation
- Editable translation layers
- Searchable metadata
- Non-destructive editing
- Multi-language support

The system must treat translations as editable metadata rather than modifying the original image.

The primary target use case is manga and comics, but the architecture should support screenshots, documents, webtoons, game dialogue, visual novels, and scanned books.

Core Design Principle

Traditional translation systems:

```
Image
-> OCR
-> Translation
-> Burn text into image
-> Done
```

This destroys information.

The platform shall instead operate as:

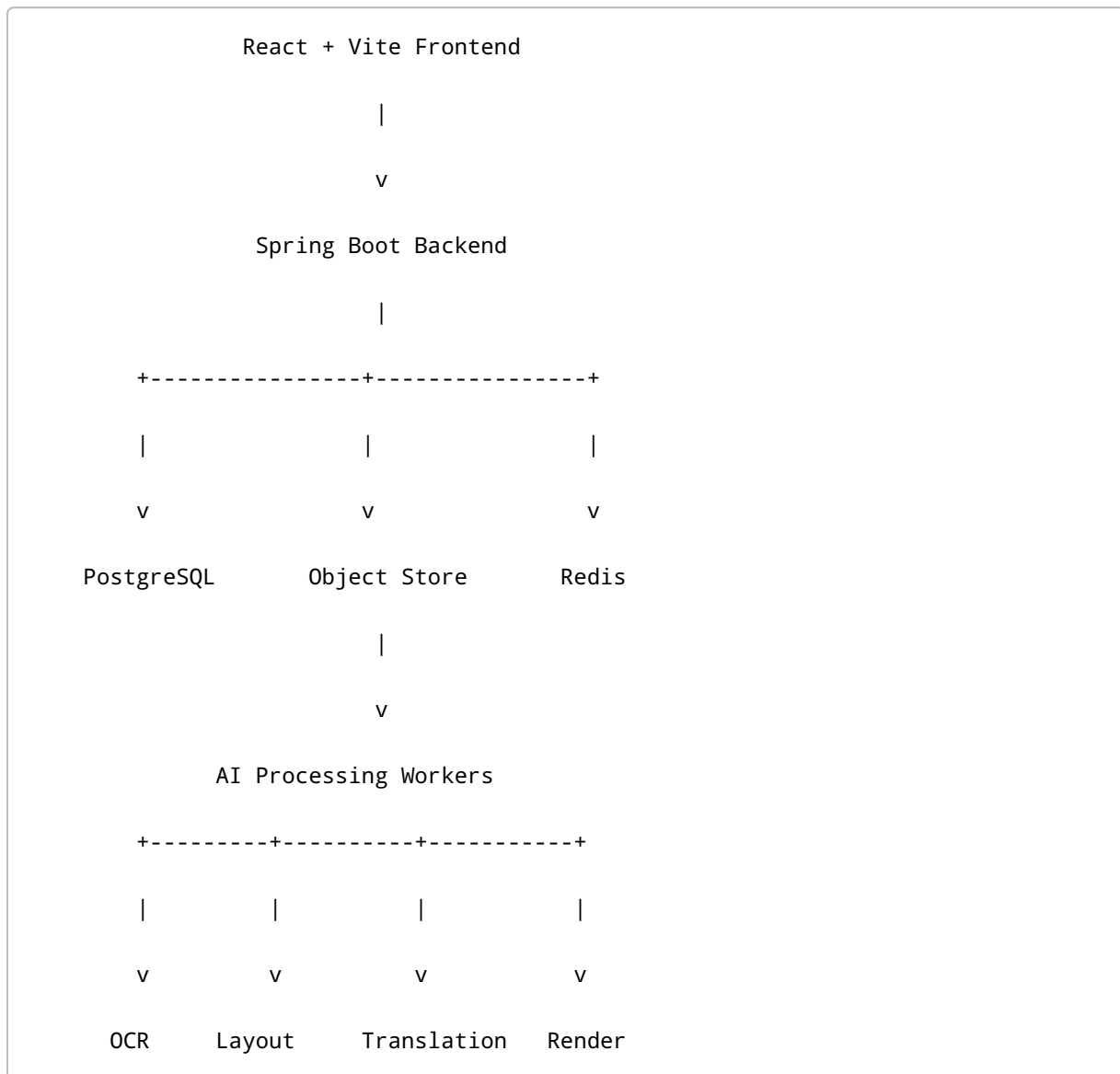
```
Image
-> OCR
-> Layout Analysis
-> Context Analysis
-> Translation
```

- > Translation Layers
- > Render View

The original image remains untouched.

Every stage produces reusable metadata.

System Architecture



Development Roadmap

Phase 1

OCR Foundation

Goal:

Detect text and bounding boxes reliably.

Deliverable:

ocr_page.jpg

Outputs:

page.ocr.json
page.overlay.png

Requirements

Detect:

- Speech bubbles
- Narration boxes
- UI text
- Screenshots
- Vertical Japanese text
- Horizontal English text

Technology

Detection:

PaddleOCR

Recognition:

MangaOCR

Hybrid pipeline:

```
Image
  ↓
PaddleOCR
  ↓
Bounding Boxes
  ↓
MangaOCR
  ↓
Text Recognition
```

Canonical OCR Schema

```
{
  "imageId": "page001",

  "regions": [
    {
      "id": "r001",

      "text": "やばい",

      "bbox": {
        "x": 123,
        "y": 200,
        "width": 80,
        "height": 90
      },

      "rotation": 90,

      "confidence": 0.98
    }
  ]
}
```

Raw OCR output must never be stored directly.

All OCR engines must map into this schema.

Reading Order Detection

OCR order is insufficient.

Need:

```
Bubble 1  
Bubble 2  
Bubble 3
```

rather than random OCR order.

Add:

```
{  
  "readingOrder": 1  
}
```

Conversation Detection

Critical requirement.

Do NOT translate individual text regions.

Group them first.

Example:

```
Bubble A  
Bubble B  
Bubble C
```

becomes:

```
{  
  "conversationId": "conv001",  
  
  "regions": [  
    "r1",  
    "r2",  
  ]  
}
```

```
    "r3"  
  ]  
}
```

This becomes the fundamental translation unit.

Phase 2

Layout Understanding

Goal:

Transform OCR regions into meaningful structure.

Detect

- Speech bubbles
 - Thought bubbles
 - Narration boxes
 - Sound effects
 - Signs
 - Device screens
 - Captions
-

Output

```
{  
  "regionId": "r001",  
  
  "type": "speech"  
}
```

Possible values:

```
speech  
thought  
narration  
sfx  
caption
```

```
sign  
ui
```

Speaker Association

Where possible:

```
{  
  "speakerId": "char001"  
}
```

This becomes important for consistency later.

Phase 3

Context-Aware Translation Engine

This phase differentiates the platform from OCR translators.

Fundamental Principle

Translation cache must NOT use source text.

This is incorrect:

```
{  
  "やばい": "Oh no"  
}
```

The same phrase may have dozens of meanings.

Translation Unit

Translation input:

```
Page Image  
+
```

```
OCR Regions
+
Reading Order
+
Conversation Grouping
+
Page Context
+
Chapter Context
```

Translation Pipeline

Step 1

Assemble context package.

```
{
  "page": "...",
  "conversations": [...],
  "previousPage": "...",
  "chapterSummary": "..."
}
```

Step 2

Send to VLM.

Candidate models:

```
GPT-5o
Gemini
Qwen-VL
Future local VLMs
```

Step 3

Request structured output.

Prompt objectives:

- Understand emotions
 - Understand scene
 - Understand relationships
 - Preserve intent
 - Preserve tone
 - Preserve jokes
 - Preserve honorifics
-

Translation Output

```
{
  "conversationId": "conv001",
  "translations": [
    {
      "regionId": "r001",
      "source": "やばい",
      "translation": "Oh no..."
    }
  ]
}
```

Translation Metadata

Store:

```
{
  "emotion": "embarrassed",
  "confidence": 0.91,
  "speaker": "female_lead",
  "tone": "romantic"
}
```

Future improvements may leverage this.

Phase 4

Translation Layer System

Most important architectural decision.

Rule

Translations must never modify original images.

Layer Model

```
{
  "layerId": "layer001",

  "type": "translation",

  "visible": true,

  "elements": [
    {
      "regionId": "r001",

      "text": "Oh no..."
    }
  ]
}
```

Editable Properties

Each layer element supports:

```
{
  "text": "...",

  "font": "...",

  "size": 20,

  "rotation": 0,
```

```
"x": 100,  
"y": 200,  
"visible": true  
}
```

Photoshop Analogy

```
Base Image  
OCR Layer  
Translation Layer  
Notes Layer  
Mask Layer
```

Everything remains editable.

Phase 5

Storage Architecture

Object Store

Recommended:

```
MinIO
```

Compatible with:

```
AWS S3  
Backblaze  
Cloudflare R2
```

Bucket Structure

originals/

Original images.

ocr/

OCR output.

layout/

Layout metadata.

translations/

Translation results.

layers/

Layer definitions.

renders/

Preview renders.

Example

originals/page001.jpg

ocr/page001.json

layout/page001.json

```
translations/page001.en.json
```

```
layers/page001.json
```

```
renders/page001.preview.jpg
```

Phase 6

PostgreSQL Schema

Images

```
images
```

Fields:

```
id  
filename  
width  
height  
hash  
created_at
```

OCR Regions

```
ocr_regions
```

Fields:

```
id  
image_id  
text  
confidence  
rotation
```

```
reading_order  
bbox
```

Conversations

```
conversations
```

Fields:

```
id  
image_id
```

Conversation Regions

```
conversation_regions
```

Fields:

```
conversation_id  
region_id
```

Translations

```
translations
```

Fields:

```
id  
conversation_id  
language  
translation_json  
model  
created_at
```

Layers

layers

Fields:

id
image_id
type
visible
data_json

Search Index

search_index

Stores:

OCR text
Translations
Tags
Metadata

Phase 7

Spring Boot Backend

Modules

image-service

ocr-service

layout-service

translation-service

layer-service
search-service
render-service

Upload Flow

Upload Image
↓
Store Original
↓
Queue OCR
↓
Queue Layout
↓
Queue Translation
↓
Create Layers

API

Upload:

`POST /api/images`

OCR:

`POST /api/images/{id}/ocr`

Translation:

`POST /api/images/{id}/translate`

Layers:


```
GET /api/images/{id}/layers
```

Update Layer:

```
PUT /api/layers/{id}
```

Search:

```
GET /api/search?q=...
```

Async Processing

Use queues.

Never block HTTP.

Recommended:

```
Redis
```

Workers process jobs asynchronously.

Phase 8

Frontend

Technology:

```
React  
Vite  
TypeScript  
TanStack Query  
Konva
```

Gallery View

Immich-inspired.

```
+-----+-----+-----+
|img |img |img |
+-----+-----+-----+
```

Features:

- Infinite scrolling
- Fast thumbnails
- Search
- Collections

Viewer

Modes:

```
Original
OCR
Translation
Split View
Layer View
```

OCR Overlay

Shows:

- Bounding boxes
- Confidence
- Reading order

Translation Overlay

Shows:

- Translated text
- Editable positions
- Editable fonts

Layer Panel

- ☒ OCR
- ☒ Translation
- ☐ Notes
- ☐ Masks

Editing Workflow

Click region.

Edit:

Translation
Font
Size
Position
Rotation
Visibility

Save instantly.

Phase 9

Search

Immich-style search is a major feature.

Search must include:

OCR text
Translations
Metadata
Tags

Example

User searches:

festival

Finds:

- OCR text
 - English translation
 - Future semantic matches
-

Phase 10

Rendering Engine

Export Formats

PNG

JPEG

WEBP

PDF

CBZ

Export Modes

Original:

Image only

Translation:

Image + translation layer

Bilingual:

Original + translation

Future Phases

Chapter Understanding

Context across pages.

Page 1
Page 2
Page 3

used during translation.

Character Memory

Track:

Names
Nicknames
Honorifics
Speech patterns

across chapters.

Inpainting

Remove original text.

Generate clean bubbles.

Place translated text.

Local AI

Replace cloud VLMs with:

Qwen-VL
Future multimodal models

for self-hosted deployments.

PSD Export

Generate layered exports.

Base Layer
OCR Layer
Translation Layer
Notes Layer

for professional editing workflows.

Development Order

1. OCR Worker
2. OCR Overlay Generator
3. Reading Order Detection
4. Conversation Grouping
5. Layout Understanding
6. Translation Context Assembly
7. VLM Translation Worker
8. Translation Layer Engine
9. PostgreSQL Schema
10. Spring Boot API
11. MinIO Integration
12. React Viewer
13. Layer Editor
14. Search

- 15. Export Engine
 - 16. Chapter Context
 - 17. Character Memory
 - 18. Inpainting
 - 19. Local Model Support
-

Success Criteria

A user uploads a manga page.

The system:

- 1. Detects text.
- 2. Determines reading order.
- 3. Groups dialogue into conversations.
- 4. Understands page context.
- 5. Produces context-aware translations.
- 6. Creates editable translation layers.
- 7. Preserves the original image.
- 8. Allows manual correction.
- 9. Supports future retranslation without rerunning OCR.
- 10. Presents everything through an Immich-like gallery and viewer experience.